

As the PCA reduces the correlation between the principal components, it removes the correlated variables. This helps in computing costs and sometimes improves the performance of algorithms. It also reduces over-fitting and improves visualisation.

We have used a standard scaler before performing the PCA. This is a must before applying PCA, as it cannot find the proper principal components if the data set is not standardised. There will be some information loss if the number of principal components selected is less than the number of features, and that can compromise the computing costs and performance of the algorithm.

Applications of PCA: PCA has applications other than dimensionality reduction, such as:

- Image compression
- Facial recognition — it is the heart of the Eigen faces algorithm for facial recognition
- Clustering of data
- Anomaly detection

Linear discriminant analysis (LDA)

Linear discriminant analysis is another dimensionality reduction technique and can be applied only on labelled data. Hence, it is also a supervised dimensionality reduction technique. It concentrates on maximising the separability among classes in the training data.

LDA follows the rules given below for projecting the data sets on the new axis:

- It maximises the distance between the means across the samples of classes.
- It minimises the scatter among the samples of one class.

LDA creates a new axis such that the value $(\mu_1 - \mu_2)^2 / (s_1^2 + s_2^2)$ is maximum, where μ_1 and μ_2 are the means of the category, while s_1 and s_2 are the standard deviations of the category.

Let's take the same example of the wine data set from the *sklearn* package and apply LDA:

```
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis as LDA
lda = LDA(n_components=2) #2-dimensional LDA
lda_transformed = pd.DataFrame(lda.fit_transform(x, data
target))
# Plot all three series
pyplot.scatter(lda_transformed[data.target==0][0], lda_
transformed[data.target==0][1], label= 'Class 1' , c= 'red'
pyplot.scatter(lda_transformed[data.target==1][0], lda_
transformed[data.target==1][1], label= 'Class 2' = 'blue'
pyplot.scatter(lda_transformed[data.target==2][0], lda_
transformed[data.target==2][1], label= 'Class 3'= 'lightgreen'
)
# Display Legend and show plot
```

```
pyplot.title('2 component LDA', fontsize = 15)
pyplot.xlabel('LD component 1', fontsize = 15)
pyplot.ylabel('LD component 2', fontsize = 15)
pyplot.legend(loc=3)
pyplot.show()
```

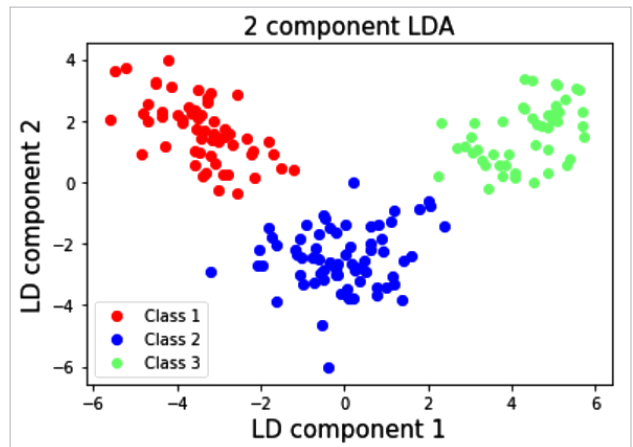


Figure 3: LDA for wine data set

We see from the plot that the individual classes are separated from each other using LDA rather than PCA. This is because the LDA maximises the separation between the classes, whereas PCA creates the components such that the variances of the samples are maximised. PCA is an unsupervised learning algorithm, whereas LDA is a supervised learning algorithm.

Applications of LDA: LDA can be used for classification problems. It can be used to predict which class the new sample belongs to. For example, if you have attributes for classifying a disease, then we can classify a new sample to check if it belongs to the disease. LDA can also be used for pattern recognition — for example, for detecting an object like a door, tyre, etc.

The t-distributed stochastic neighbour embedding (t-SNE)

The t-distributed stochastic neighbour embedding (t-SNE) is an unsupervised non-linear technique developed by Laurens van der Maaten and Geoffrey Hinton in 2008, which can be used for exploring high dimensional data. It does dimensionality reduction for data visualisation. t-SNE not only captures the local structure of the higher dimension but also preserves the global structures of the data such as clusters. The t-SNE is based on the stochastic neighbour embedding algorithm (SNE) and is an improvement on the latter.

Let us understand the SNE algorithm in brief. Let us assume there are sets of data points in higher dimensions, which need to be mapped to lower dimensions while preserving the neighbourhood structure of the data set. The